

ViolationIQ: An Adaptive Multi-Expert AI Evidence Copilot for Traffic Violation Evidence Generation

Hrithik Sharma

Computer Vision Based Traffic Enforcement Evidence Generation System Problem Statement: Automated Photo Identification and Classification

Abstract—Traffic surveillance systems generate a large number of road images and videos every day. Manual inspection of this data for traffic violations is slow, inconsistent, and difficult to scale. This report presents ViolationIQ, an adaptive multi-expert computer vision system for traffic violation evidence generation. The system is designed as an AI evidence copilot, not as an automatic challan generator. It processes selected traffic images and videos to generate clean, review-ready evidence for rider-wise helmet violation detection, number plate localization, safe OCR/ANPR, and signboard context detection. The implemented prototype uses YOLO11s-based expert models, OpenCV-based image and video processing, EasyOCR with Indian number plate validation rules, structured JSON/CSV reporting, and a safety layer that routes uncertain cases to manual review. The final judge-reproducible package generates five helmet/rider evidence images, five safe OCR evidence panels, five signboard context evidence images, one processed helmet violation demo video, JSON reports, a final CSV index, and a reproduction summary. The main focus of ViolationIQ is evidence reliability, false challan prevention, reviewability, and reproducibility.

Index Terms—Computer Vision, Traffic Violation Detection, YOLO, Helmet Detection, Number Plate Detection, OCR, ANPR, Signboard Detection, Intelligent Transportation Systems, Evidence Generation.

I. INTRODUCTION

Traffic enforcement agencies increasingly rely on CCTV cameras, road monitoring systems, and surveillance infrastructure. These systems continuously produce traffic images and videos. However, reviewing this evidence manually is time-consuming and difficult when camera feeds are large in number. Automated computer vision can reduce this workload, but traffic enforcement is a sensitive domain because incorrect automated decisions may lead to wrong penalties.

ViolationIQ follows a safety-first design. It is not designed to directly issue challans. Instead, it assists human reviewers by generating clean, structured, and review-ready evidence. The system detects relevant visual objects, converts them into evidence, and marks uncertain cases for manual review.

The proposed system uses multiple specialized expert modules instead of one monolithic model. Different traffic evidence tasks require different reasoning. Helmet violations require rider-wise association. Plate OCR requires plate localization and text validation. Signboard evidence requires road-rule context. Video evidence requires frame-level processing. Hence, ViolationIQ uses an adaptive multi-expert architecture.

II. MOTIVATION

Traffic violation detection is not only an object detection problem. A practical traffic enforcement system must produce evidence that can be reviewed by a human officer. For example, a helmet violation may be verified from a clear frame, but number plate OCR may fail when the plate is small, blurred, tilted, or overexposed. Similarly, a signboard may provide traffic rule context, but final enforcement may still require human verification.

This motivated the design of ViolationIQ as an evidence generation framework. The system focuses on:

- creating clean evidence panels,
- avoiding forced OCR output,
- providing structured reports,
- supporting manual review,
- improving reproducibility for judges and evaluators.

III. PROBLEM UNDERSTANDING

The problem statement requires automated photo identification and classification of traffic violations using computer vision. A complete system must answer three questions.

First, the system must identify what is visible in a traffic scene. It must detect riders, helmets, number plates, vehicles, and signboards. Second, it must interpret the meaning of the detections. For example, a rider without a helmet is a helmet violation evidence candidate, while a visible signboard may provide traffic rule context. Third, the system must decide whether the evidence is reliable enough for review or whether it should be sent to manual review.

A simple detector may draw bounding boxes, but a real enforcement workflow needs more than detection. It needs reviewer-friendly evidence, confidence values, reliability notes, JSON reports, and safety checks. ViolationIQ addresses this by combining detection, reasoning, OCR validation, and manual-review fallback.

IV. KEY CHALLENGES

The main practical challenges in traffic violation evidence generation are:

- small and blurred number plates,
- low-resolution or noisy traffic frames,
- occluded riders and helmets,

- overlapping vehicles and riders,
- camera angle variation,
- poor lighting and motion blur,
- incorrect OCR text from weak plate crops,
- need for human verification before enforcement action.

These challenges make a safety-first workflow necessary. ViolationIQ therefore avoids unsafe automatic conclusions and marks doubtful cases for manual review.

V. PROPOSED SYSTEM

ViolationIQ is a modular traffic evidence generation system. It accepts selected traffic images and videos as input and produces annotated evidence images, a processed demo video, JSON reports, CSV summaries, and reproduction metadata.

The final implemented system includes:

- adaptive multi-expert routing,
- helmet and rider evidence generation,
- processed helmet demo video generation,
- number plate localization,
- safe OCR/ANPR evidence generation,
- signboard context evidence generation,
- JSON and CSV report generation,
- judge reproduction script.

The system is designed as an AI evidence copilot. It supports review and decision-making, but it does not automatically generate final legal challans.

VI. SYSTEM ARCHITECTURE

The architecture of ViolationIQ is divided into five main layers.

A. Input Layer

The input layer accepts selected traffic images and videos. In the final reproducible package, the inputs include OCR showcase images, signboard showcase images, and a raw clean helmet demo video.

B. Adaptive Routing Layer

The adaptive routing layer decides which expert module should process the input. Image inputs may be processed by the OCR module or signboard context module. Video input is processed by the helmet video evidence module.

C. Expert Module Layer

The expert module layer contains specialized modules:

- Helmet and rider expert,
- Plate and safe OCR expert,
- Signboard context expert,
- Helmet video expert.

D. Evidence Reasoning Layer

The evidence reasoning layer converts raw detections into meaningful outputs. It performs rider-wise association, plate text validation, signboard context interpretation, and report generation.

E. Safety Layer

The safety layer prevents unsafe conclusions. It sends unclear evidence, weak OCR, low-confidence detections, and ambiguous cases to manual review.

VII. DESIGN PHILOSOPHY

ViolationIQ follows four important design principles.

A. Modularity

Each traffic evidence task is handled by a specialized expert module. This makes the system easier to debug, extend, and evaluate.

B. Explainability

The system does not only output labels. It generates evidence panels, status messages, confidence values, and JSON reports.

C. Safety

The system does not force weak OCR or uncertain detections into final decisions. Manual review is used whenever evidence is not reliable.

D. Reproducibility

The project includes a source code zip and a separate Judge Asset Zip. This allows judges to reproduce the final showcase outputs without storing large files inside the source code package.

VIII. DATASETS AND ASSETS

The project uses task-specific datasets and selected final assets.

For the helmet and rider module, helmet detection data and a final raw bike demo video are used. The video is used to generate the processed helmet violation demo output. For plate detection and OCR, a dedicated plate detection model and selected OCR showcase images are used. For signboard context, selected real signboard images with XML annotations are used.

The Judge Asset Zip contains the trained model weights and sample inputs required for reproduction. After extraction, the repository contains the following important files and folders:

```
weights/traffic_yololls_best.pt
weights/helmet_yololls_best.pt
weights/large_plate_yololls_best.pt
weights/yolo26n.pt
```

```
data/sample_images/ocr_showcase/
data/sample_images/signboard_showcase/
data/sample_videos/violationiq_bike_helmet_demo.mp4
```

Large files are not kept inside the source code zip. They are provided separately through the Judge Asset Zip.

IX. TECHNOLOGY STACK

The project uses the following technologies:

- **Python:** main implementation language.
- **YOLO11s:** object detection for helmet, rider, traffic, and number plate modules.
- **OpenCV:** image processing, video processing, frame handling, and evidence drawing.
- **EasyOCR:** number plate text extraction.
- **NumPy:** numerical processing of image arrays and coordinates.
- **JSON:** structured evidence reports.
- **CSV:** final showcase index and summary.
- **GitHub:** source code hosting and documentation.
- **Google Drive:** hosting of large judge assets.

X. MODEL RESULTS

The project uses trained YOLO-based expert models. The recorded development results are:

- Traffic YOLO11s model: precision around 0.898, recall around 0.880, mAP_{50} around 0.930, and mAP_{50-95} around 0.808.
- Helmet/rider YOLO11s model: mAP_{50} around 0.701 and mAP_{50-95} around 0.32 during development.
- Dedicated plate YOLO11s model: validation mAP_{50} around 0.924 and mAP_{50-95} around 0.548.
- Final reproduction package: generates five helmet/rider evidence images, five OCR evidence panels, five signboard context panels, and one processed helmet video.

The helmet/rider task is more difficult than simple object detection because riders may be partially visible, helmets can be small, and crowded scenes may include overlapping riders. OCR is also challenging because blurred or small number plates can produce wrong text. Therefore, the safety layer is an important part of the system.

XI. METHODOLOGY

The methodology has five stages.

A. Data Preparation

The training datasets were inspected and organized into the required format. For final judge reproduction, selected images, video, and XML annotations were placed inside the asset package.

B. Model Training

YOLO11s-based models were trained for traffic context, helmet/rider detection, and plate localization. The best trained weights were saved and used in the final reproduction pipeline.

C. Inference

During inference, images or video frames are processed by the relevant expert module. The system stores bounding boxes, class labels, confidence values, and metadata.

D. Evidence Reasoning

Raw detections are converted into evidence. For helmet evidence, rider-wise association is performed. For OCR evidence, detected plate crops are enhanced and passed through OCR. The OCR result is cleaned and validated before being shown as reliable evidence.

E. Output Generation

The final stage produces evidence images, processed demo video, JSON reports, a CSV index, and a reproduction summary.

XII. HELMET AND RIDER EVIDENCE MODULE

The helmet module detects riders and helmet-related classes. The main task is not only to detect a helmet, but also to associate helmet or no-helmet evidence with the correct rider.

For each detected rider bounding box, the system checks whether a helmet-related detection lies inside the rider region. The center point of a detected box is computed as:

$$c_x = \frac{x_1 + x_2}{2}, \quad c_y = \frac{y_1 + y_2}{2} \quad (1)$$

A detection is associated with a rider if:

$$x_r^1 \leq c_x \leq x_r^2 \quad \text{and} \quad y_r^1 \leq c_y \leq y_r^2 \quad (2)$$

The final helmet output includes:

- five helmet/rider evidence images,
- rider-wise bounding boxes,
- helmet or no-helmet status,
- confidence and review notes,
- one processed helmet violation demo video,
- JSON evidence reports.

XIII. NUMBER PLATE DETECTION AND SAFE OCR MODULE

The number plate module first localizes the plate using a dedicated detector. OCR is then applied only to the extracted plate region. This improves reliability compared with applying OCR directly to the full image.

The OCR pipeline includes:

- plate crop extraction,
- crop resizing,
- contrast enhancement,
- EasyOCR reading,
- text cleaning,
- Indian number plate format validation,
- manual-review fallback.

ViolationIQ does not force OCR output when the text is weak. If OCR confidence is low or the text format is not reliable, the system marks the case for manual review. This reduces the risk of incorrect plate reporting.

The OCR pipeline is CPU-compatible for judge reproducibility. GPU can make OCR faster, but the same validation logic and manual-review fallback are used.

XIV. SIGNBOARD CONTEXT EVIDENCE MODULE

The signboard context module uses selected real signboard images and XML annotations. It generates evidence panels showing signboard type, bounding box, context meaning, and review note.

Supported signboard contexts include:

- stop sign,
- traffic light sign,
- speed-limit sign,
- turn restriction sign,
- no-entry or restriction context where available.

The signboard module does not directly issue a violation. It provides road-rule context evidence. Final enforcement may require additional vehicle direction or tracking evidence depending on the rule.

The final signboard output includes:

- five signboard context evidence images,
- XML annotation based bounding boxes,
- signboard class labels,
- signboard JSON reports,
- signboard summary report.

XV. SAFETY LAYER

Traffic enforcement systems must be careful because wrong automated decisions can affect citizens. Therefore, ViolationIQ uses a safety-first workflow.

Cases are routed to manual review when:

- OCR confidence is weak,
- plate crop is unreadable,
- detection confidence is low,
- rider association is ambiguous,
- signboard context alone is insufficient,
- visual evidence is unclear.

The system uses evidence labels such as reliable, candidate, or manual review. This makes the system a decision-support tool rather than a final legal authority.

XVI. EVIDENCE OUTPUT DESIGN

The evidence output is designed for human reviewers.

The helmet evidence panel contains the traffic frame, rider-wise boxes, rider IDs, helmet status, confidence values, and review notes. The OCR evidence panel contains the original image, detected plate region, enhanced plate crop, OCR output or candidate text, validation status, and manual-review warning. The signboard context panel contains the original signboard image, sign bounding box, detected sign class, context explanation, and review note.

This output design makes the system more practical than a simple detector because it provides review-ready evidence instead of only raw predictions.

XVII. FINAL IMPLEMENTED DELIVERABLES

The final implemented deliverables are:

- Adaptive multi-expert routing system.
- Helmet and rider evidence generation.
- Five rider-wise helmet/no-helmet evidence images.
- One processed helmet violation MP4 demo video.
- Dedicated number plate detection.
- Five safe OCR evidence panels with validation and manual-review fallback.
- Five signboard context evidence images using selected images and XML annotations.
- JSON reports for helmet, OCR, signboard, and video modules.
- Final showcase CSV index.
- Reproduction summary JSON.
- Source code zip for HackerEarth submission.
- Judge Asset Zip containing weights and sample inputs.

XVIII. REPOSITORY AND OUTPUT STRUCTURE

The final repository contains code, scripts, documentation, reports, and lightweight output metadata.

```
ViolationIQ/
|-- README.md
|-- requirements.txt
|-- REPRODUCE_FOR_JUDGES.md
|-- REPRODUCIBILITY_SCOPE.md
|-- src/
|   |-- adaptive_router.py
|   |-- helmet_plate_module.py
|   |-- safety_utils.py
|   |-- signboard_context_module.py
|-- scripts/
|   |-- reproduce_pipeline.py
|   |-- process_ocr_samples.py
|   |-- process_helmet_video_demo.py
|   |-- process_signboard_showcase.py
|   |-- download_assets.py
|-- outputs/
|   |-- FINAL_SHOWCASE/
|       |-- helmet_plate/
|       |-- plate_ocr/
|       |-- signboard_context/
|       |-- videos/
|       |-- final_showcase_index.csv
|-- reports/
|   |-- reproduction_summary.json
```

XIX. JUDGE REPRODUCIBILITY

The project is designed to be reproduced by judges. The source code zip contains code, scripts, documentation, and lightweight reports. The Judge Asset Zip contains trained weights, selected OCR showcase images, selected signboard

showcase images, XML annotations, and the raw clean helmet demo video.

Repository and asset links:

- GitHub Repository: <https://github.com/Hrithikcrick/ViolationIQ>
- Judge Asset Zip: <https://drive.google.com/file/d/1LWZj-UicQOj7A8Tgj7nxW3Cm-2gn7EOJ/view?usp=sharing>

The final reproduction command is:

```
python scripts\reproduce_pipeline.py --traffic_model_weights\traffic_volo11s_best.pt --helmet_
```

The expected output is:

```
Reproduction completed.  
Helmet evidence images generated: 5  
Signboard context images generated: 5  
Safe OCR reports generated: 5  
Helmet videos processed: 1  
Summary saved: reports/reproduction_summary.json  
Showcase index: outputs/FINAL_SHOWCASE/final_showcase_index.csv
```

XX. MAPPING TO PROBLEM STATEMENT

ViolationIQ addresses the main requirements of the problem statement as follows:

- It processes traffic images and videos using Python and OpenCV.
- It detects road users, helmets, plates, and signboards using YOLO-based expert models.
- It identifies helmet violation evidence and signboard context evidence.
- It performs number plate localization and OCR using EasyOCR with validation rules.
- It generates annotated evidence panels with boxes, labels, status, and review notes.
- It saves JSON reports, CSV index, and reproduction summary.
- It avoids unsafe automatic challans using manual-review fallback.
- It supports judge reproducibility using source code and a separate asset zip.

XXI. DEPLOYMENT WORKFLOW

In a practical deployment scenario, ViolationIQ can be used as follows:

- 1) A traffic image or video is collected from a camera.
- 2) The adaptive router identifies the suitable expert module.
- 3) The selected expert processes the image or video.
- 4) The reasoning layer converts detections into evidence.
- 5) The safety layer checks confidence and reliability.
- 6) Evidence panels and JSON/CSV reports are generated.
- 7) A human reviewer verifies the evidence before enforcement action.

This workflow reduces manual workload while keeping the final decision under human supervision.

XXII. LIMITATIONS

The current prototype focuses on judge-reproducible selected showcase outputs rather than full city-scale deployment. OCR performance can degrade on very blurred or low-resolution plates. Helmet detection can be affected by occlusion, overlapping riders, and camera angle. Signboard context alone may not always prove a violation unless combined with vehicle movement or direction evidence.

The project currently emphasizes evidence generation and reviewability. Full deployment would require camera calibration, audit logging, privacy safeguards, officer dashboard integration, and periodic model validation.

XXIII. FUTURE SCOPE

Future improvements can include:

- improved Indian plate OCR model,
- stronger low-light enhancement,
- multi-camera tracking,
- real-time dashboard for traffic officers,
- temporal tracking for wrong-side and red-light violations,
- automatic evidence clustering from multiple camera feeds,
- active learning from reviewer feedback,
- privacy-preserving masking for non-relevant persons.

XXIV. CONCLUSION

This report presented ViolationIQ, an adaptive multi-expert AI evidence copilot for traffic violation evidence generation. The system processes selected traffic images and videos to generate helmet/rider evidence, safe OCR/ANPR evidence, signboard context evidence, processed helmet demo video, JSON reports, CSV index, and reproduction summary.

The final prototype uses YOLO11s-based detection, OpenCV processing, EasyOCR-based OCR, Indian number plate validation rules, and a safety-first manual-review layer. The system is not an automatic challan generator. It is designed to support traffic enforcement teams by producing clean, structured, and review-ready evidence while avoiding unsafe conclusions in uncertain cases.

The project is also judge-reproducible through a lightweight source code zip and a separate Judge Asset Zip containing trained weights and sample inputs. This makes the prototype practical for evaluation, demonstration, and future extension.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [2] G. Bradski, "The OpenCV Library," *Dr. Dobbs's Journal of Software Tools*, 2000.
- [3] J. Memon, M. Sami, R. A. Khan, and M. Uddin, "Handwritten Optical Character Recognition: A Comprehensive Systematic Literature Review," *IEEE Access*, vol. 8, pp. 142642–142668, 2020.
- [4] M. A. Boukerche and J. Wang, "Machine Learning-Based Traffic Prediction Models for Intelligent Transportation Systems," *Computer Networks*, vol. 181, 2020.
- [5] Ultralytics, "YOLO Documentation," 2024. [Online]. Available: <https://docs.ultralytics.com/>
- [6] JaidedAI, "EasyOCR," 2024. [Online]. Available: <https://github.com/JaidedAI/EasyOCR>